JPA vs Hibernate vs Spring Data JPA

JPA (Java Persistence API)

JPA is **not a tool or library that does work** — it's a **specification** (a rulebook) defined by Java that describes how Java objects should be mapped to database tables. It defines annotations like `@Entity`, `@Id`, and `@GeneratedValue`, and interfaces like `EntityManager`. On its own, JPA has no actual code that talks to a database — it just describes *how* persistence should work. Think of it as a job description with no employee hired yet.

Hibernate

Hibernate is the most popular **implementation** of the JPA specification. It's the actual engine that reads your `@Entity`-annotated classes and JPA annotations, and translates them into real SQL statements (`INSERT`, `SELECT`, `UPDATE`, `DELETE`) that run against the database. When a JPA method like `entityManager.persist(book)` is called, Hibernate is the one silently generating and executing the SQL behind the scenes. It's the employee actually doing the job that JPA's rulebook describes.

Spring Data JPA

Spring Data JPA is a layer built **on top of** JPA and Hibernate, designed to remove almost all boilerplate code. Without it, even with Hibernate, a developer must still manually write code such as:

```
entityManager.persist(book);

entityManager.createQuery("SELECT b FROM Book b").getResultList();
```

With Spring Data JPA, the same functionality is achieved by just declaring an interface:

```
public interface BookRepository extends JpaRepository<Book, Long> {

}
```

No method bodies are written. Spring Data JPA automatically generates a working implementation at runtime, providing methods like **`save()`**, **`findAll()`**, **`findById()`**, and **`deleteById()`** for free.

Simple Analogy

Layer	Analogy
JPA	The rulebook describing how translation should work
Hibernate	The human translator who actually does the translation
Spring Data JPA	A translation app — you just say what you want, it handles everything underneath

How This Connects to the Hands-On Example

In the **`orm-learn`** project built for this exercise:

- `Book.java` uses **JPA annotations** (`@Entity`, `@Id`, `@GeneratedValue`) to define how the class maps to a database table.
- **Hibernate** (used automatically by Spring Boot underneath) converts that class definition into an actual `book` table in the H2 database and generates the SQL to insert/fetch rows.
- `BookRepository.java` uses **Spring Data JPA** (`extends JpaRepository<Book, Long>`) to get fully working **`save()`** and **`findAll()`** methods without writing any SQL or implementation code.

Running the application confirmed all three layers working together: a book was saved and successfully retrieved from the database with zero manual SQL.